Problem Formulation

Our Matlab library is designed to solve constrained optimization problems of the form

$$\begin{aligned} &\min_{u,z} J(u,z) \\ &s.t. \quad c(u,z) = 0 \end{aligned} \tag{1}$$

via a reduced space optimization strategy. In particular, if $c(u,z) = 0$ admits a unique solution for any given z, then there exists a solution operator $S(z)$ such that $c(S(z),z) = 0 \; \forall z$. We may reformulate (1) in reduced space and solve the equivalent optimization problem

$$\min_{z} \hat{J}(z) = J(S(z),z) \tag{2}$$

which will be the focus of this document and the corresponding Matlab codes. Along with solving (2), we would also like the code to facilitate the use of hyper-differential sensitivity analysis (HDSA) for the solution of (2).

Base Class: *Constrained_Optimization*

We adopt an object oriented design in Matlab by defining a base class *Constrained_Optimization* where the problem, optimization, and HDSA interfaces will be defined. As we present the mathematics used in *Constrained_Optimization* we will introduce functions, properties, and inherited classes corresponding to the mathematical expressions.

To facilitate our analysis, we require efficient algorithms to compute the gradient $\nabla_z \hat{J}(z)$ and hessian vector products $\nabla_{z,z} \hat{J}(z)v$, for a given vector v. To this end we introduce adjoint-based derivatives which is implemented in the code. Algorithm 1 demonstrates how to efficiently calculate the gradient. In the algorithm and throughout this document we will use c_u and c_z to denote the Jacobians of c with respect to u and z, respectively, $\nabla_u J$ and $\nabla_z J$ to denote the gradients (where u and z are independent of one another) of J with respect to u and z, respectively, and a superscript T to denote a matrix transpose. Based on Algorithm 1, Table 1 summarizes pure virtual functions defined in *Constrained_Optimization* which must be implemented for a given problem to enable the gradient computation.

Algorithm 1 Adjoint-based gradient calculation

Input: $\overline{z}$
1: Solve the state equation to determine $\overline{u}$ such that $c(\overline{u},\overline{z}) = 0$
2: Solve the adjoint equation $c_u(\overline{u},\overline{z})^T\overline{\lambda} = -\nabla_u J(\overline{u},\overline{z})$ for $\overline{\lambda}$ (i.e. a linear solve)
3: Compute $\nabla_z \hat{J}(\overline{z}) = c_z(\overline{u},\overline{z})^T\overline{\lambda} + \nabla_z J(\overline{u},\overline{z})$
Return: $\nabla_z \hat{J}(\overline{z})$

Function Name	Mathematical Description
[val, grad_u, grad_z] = Objective(obj,u,z)	Evaluate J, $\nabla_u J$, and $\nabla_z J$
[u] = State_Solve(obj,z)	Solve $c(u,z) = 0$ for u
[Mv] = c_u_Transpose_Inverse_Apply(obj,v,u,z)	Compute the product $c_u(u,z)^{-T}v$
[Mv] = c_z_Transpose_Apply(obj,v,u,z)	Compute the product $c_z(u,z)^T v$

Table 1: Summary of pure virtual function in *Constrained_Optimization* needed to execute Algorithm 1.

To compute efficient hessian vector products, Algorithm 2 outlines computation using incremental state and incremental adjoint equations. In the algorithm and throughout this document we will use $\lambda^T c_{u,u}$ and $\nabla_{u,u} J$ to denote the (u,u) hessian of the scalar functions $\lambda^T c$ and J, respectively, and similar expressions for the (u,z), (z,u), and (z,z) hessians. Table 2 summarizes pure virtual functions defined in *Constrained_Optimization* which must be implemented for a given problem to enable the hessian vector product computation using Algorithm 2.

Algorithm 2 Adjoint-based hessian vector product calculation

Input: v, $\overline{u}$, $\overline{z}$, $\overline{\lambda}$
1: Compute $w = c_z(\overline{u},\overline{z})v$
2: Solve the incremental state equation $c_u(\overline{u},\overline{z})\overline{\mu} = -w$ for $\overline{\mu}$ (i.e. a linear solve)
3: Compute $y_J = \nabla_{u,u}J(\overline{u},\overline{z})\overline{\mu} + \nabla_{u,z}J(\overline{u},\overline{z})v$
4: Compute $y_c = \overline{\lambda}^T c_{u,u}(\overline{u},\overline{z})\overline{\mu} + \overline{\lambda}^T c_{u,z}(\overline{u},\overline{z})v$
5: Solve the incremental adjoint equation $c_u(\overline{u},\overline{z})^T\overline{\gamma} = -(y_J + y_c)$ for $\overline{\gamma}$ (i.e. a linear solve)
6: Compute $x_J = \nabla_{z,u}J(\overline{u},\overline{z})\overline{\mu} + \nabla_{z,z}J(\overline{u},\overline{z})v$
6: Compute $x_c = c_z(\overline{u},\overline{z})^T\overline{\gamma} + \overline{\lambda}^T c_{z,u}(\overline{u},\overline{z})\overline{\mu} + \overline{\lambda}^T c_{z,z}(\overline{u},\overline{z})v$
6: Compute $\nabla_{z,z}\hat{J}(\overline{z})v = x_J + x_c$
Return: $\nabla_{z,z}\hat{J}(\overline{z})v$

Function Name	Mathematical Description
[Mv] = c_u_Inverse_Apply(obj,v,u,z)	Compute the product $c_u(u,z)^{-1}v$
[Mv] = c_z_Apply(obj,v,u,z)	Compute the product $c_z(u,z)v$
[Mv] = c_uu_Apply(obj,v,u,z,lambda)	Compute the product $\lambda^T c_{u,u}(u,z)v$
[Mv] = c_uz_Apply(obj,v,u,z,lambda)	Compute the product $\lambda^T c_{u,z}(u,z)v$
[Mv] = c_zu_Apply(obj,v,u,z,lambda)	Compute the product $\lambda^T c_{z,u}(u,z)v$
[Mv] = c_zz_Apply(obj,v,u,z,lambda)	Compute the product $\lambda^T c_{z,z}(u,z)v$
[Mv] = Objective_uu_Apply(obj,v,u,z)	Compute the product $\nabla_{u,u}J(u,z)v$
[Mv] = Objective_uz_Apply(obj,v,u,z)	Compute the product $\nabla_{u,z}J(u,z)v$
[Mv] = Objective_zu_Apply(obj,v,u,z)	Compute the product $\nabla_{z,u}J(u,z)v$
[Mv] = Objective_zz_Apply(obj,v,u,z)	Compute the product $\nabla_{z,z}J(u,z)v$

Table 2: Summary of pure virtual function in *Constrained_Optimization* needed to execute Algorithm 2.

We implement Algorithm 1 and Algorithm 2 in *Constrained_Optimization*'s private functions *Jhat* and *Jhat_hessVec*, respectively, and interface them with Matlab's solver *fminunc* in the public function *Optimize*. In addition, *Constrained_Optimization* has the public functions *Finite_Difference_Gradient_Check* and *Finite_Difference_Hessian_Check* to test the accuracy of our calculation of $\nabla_z\hat{J}(z)$ and $\nabla_{z,z}\hat{J}(z)v$.

Derived Class: *Constrained_ODE_Optimization*

We specialize the general optimization problem (1) and consider the case where $c(u,z) = 0$ corresponds to the discretization of a system of ordinary differential equations (ODEs)

$$\frac{du}{dt} = f(u,z,t)$$
$$u(0) = h(z)$$

where $u : [0,T] \to \mathbb{R}^m$, $T > 0$, $z \in \mathbb{R}^n$, $f : \mathbb{R}^m \times \mathbb{R}^n \times [0,T] \to \mathbb{R}^m$, and $h : \mathbb{R}^n \to \mathbb{R}^m$, and the objective function is

$$J(u,z) = \int_0^T g(u(t),t)dt + R(z)$$

where $g : \mathbb{R}^m \times \mathbb{R} \to \mathbb{R}$ and $R : \mathbb{R}^n \to \mathbb{R}$ are user specified functions.

To implement ODE constrained optimization, we derives off the base class *Constrained_Optimization* and implements its pure virtual functions in the subclass *Constrained_ODE_Optimization*. The subclass inputs the state dimension m, the final time T, and the number of time steps N, and requires the user to specify the objective function and ODE system through the pure virtual functions summarized in Table 3.

Function Name	Mathematical Description
[val, grad_u] = Time_Instance_Objective(obj,u,t)	Evaluate $g(u,t)$ and $\nabla_u g(u,t)$
[val,grad_z] = Regularization_Objective(obj,z)	Evaluate $R(z)$ and $\nabla_z R(z)$
[f, f_u, f_z] = Time_Instance_RHS(obj,u,z,t)	Evaluate $f(u,z,t)$, $f_u(u,z,t)$, and $f_z(u,z,t)$
[h, h_z] = Initial_Condition(obj,z)	Evaluate $h(z)$ and $h_z(z)$
[Mv] = Time_Instance_Objective_uu_Apply(obj,v,u,t)	Compute the product $\nabla_{u,u} g(u,t)v$
[Mv] = Regularization_Objective_zz_Apply(obj,v,z)	Compute the product $\nabla_{z,z} R(z)v$
[Mv] = Time_Instance_RHS_uu_Apply(obj,v,u,z,t,lambda)	Compute the product $\lambda^T f_{u,u}(u,z,t)v$
[Mv] = Time_Instance_RHS_uz_Apply(obj,v,u,z,t,lambda)	Compute the product $\lambda^T f_{u,z}(u,z,t)v$
[Mv] = Time_Instance_RHS_zu_Apply(obj,v,u,z,t,lambda)	Compute the product $\lambda^T f_{z,u}(u,z,t)v$
[Mv] = Time_Instance_RHS_zz_Apply(obj,v,u,z,t,lambda)	Compute the product $\lambda^T f_{z,z}(u,z,t)v$
[Mv] = Initial_Condition_zz_Apply(obj,v,z,lambda)	Compute the product $\lambda^T h_{z,z}(z)v$

Table 3: Summary of pure virtual function in *Constrained_ODE_Optimization*.

In order to implement the pure virtual functions from the base class *Constrained_Optimization* (the combinations of functions in Table 1 and Table 2), we must fix a time discretization scheme and write a finite dimensional optimization problem in the form of (1). To this end, let $t_k = T\frac{k-1}{N-1}$, $k = 1, 2, \ldots, N$ be equally spaced nodes in the time interval $[0, T]$ and $u_{i,k}$ denote the value of the i^{th} component of $u(t_k)$, i.e. the i^{th} state variable at the k^{th} time. Concatenation these discrete evaluations yields the state vector $u = (u_1, u_2, \ldots, u_N)^T \in \mathbb{R}^{mN}$ where $u_k = (u_{1,k}, u_{2,k}, \ldots, u_{m,k})^T \in \mathbb{R}^m$ denotes the state evaluated at time t_k. Then applying the backward Euler time integration scheme, we have

$$c(u,z) = \begin{pmatrix} u_1 - h(z) \\ u_2 - u_1 - \Delta t f(u_2, z, t_2) \\ u_3 - u_2 - \Delta t f(u_3, z, t_3) \\ u_4 - u_3 - \Delta t f(u_4, z, t_4) \\ \vdots \\ u_n - u_{n-1} - \Delta t f(u_N, z, t_N) \end{pmatrix} \in \mathbb{R}^{mN} \tag{3}$$

where $\Delta t = \frac{T}{N-1}$. Applying the trapezoid rule for integration, the discretized objective function is

$$J(u,z) = \sum_{k=1}^{N} w_k g(u_k, t_k) + R(z) \tag{4}$$

where $w_k = \Delta T$, $k = 2, 3, \ldots, N-1$, and $w_1 = w_N = 0.5\Delta T$, are integration weights. Given an implementation of g, R, and their derivatives, the pure virtual functions corresponding to J and its derivatives are easily implemented based on (4). The pure virtual functions corresponding to c require more careful consideration which we provide below.

To solve the equation $c(u,z) = 0$, we exploit the block structure in (3) and solve $m \times m$ systems of equations at each time point sequentially (i.e. execute time integration rather than solving $c(u,z) = 0$ as a system of mN coupled equations). However, the structure of $c(u,z)$ will be critical for ensuring that we compute the adjoints correctly. The Jacobian of c with respect to u, $c_u(u,z)$, is the $mN \times mN$ matrix

$$\begin{pmatrix} I_m & 0 & 0 & 0 & \ldots & 0 \\ -I_m & I_m - \Delta t f_u(u_2, z, t_2) & 0 & 0 & \ldots & 0 \\ 0 & -I_m & I_m - \Delta t f_u(u_3, z, t_3) & 0 & \ldots & 0 \\ 0 & 0 & -I_m & I_m - \Delta t f_u(u_4, z, t_4) & \ldots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & 0 & -I_m & I_m - \Delta t f_u(u_N, z, t_N) \end{pmatrix}.$$

Hence the implementation of $c_u(u,z)^{-T}v$ for the adjoint solve in Algorithm 1 must consider the system of linear equations $c_u(u,z)^T x = v$ written as

$$\begin{pmatrix} I_m & -I_m & 0 & 0 & \dots & 0 \\ 0 & I_m - \Delta t f_u(u_2,z,t_2) & -I_m & 0 & \dots & 0 \\ 0 & 0 & I_m - \Delta t f_u(u_3,z,t_3) & -I_m & \dots & 0 \\ 0 & 0 & 0 & I_m - \Delta t f_u(u_4,z,t_4) & -I_m & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 0 & I_m - \Delta t f_u(u_N,z,t_N) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_N \end{pmatrix} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ \vdots \\ v_N \end{pmatrix}$$

and compute $x = c_u(u,z)^{-T}v$ via the time stepping algorithm

$$\begin{aligned} x_N &= (I_m - \Delta t f_u(u_N,z,t_N))^{-1} v_N \\ x_k &= (I_m - \Delta t f_u(u_k,z,t_k))^{-1}(v_k + x_{k+1}) \qquad k = N-1, n-2, \dots, 2 \\ x_1 &= v_1 + x_2. \end{aligned}$$

This ensures that the time discretization of the adjoint equation is consistent with the time integration of the state equation at the discrete level, thus providing an accurate gradient computation. Similar care must be taken when implementing all other matrix vector products and linear solves involving derivatives of c.